CSE225L – Data Structures and Algorithms Lab
Lab 04 (Part-1)
Template Class

Task 1: Recall the class we used in the previous lab to allocate memory dynamically. Modify the header file and the source file given below so that they now work as template class (the array elements in the dynamically allocated memory can be any type as the user defines).

dynarr.h

```
#ifndef DYNARR_H_INCLUDED
#define DYNARR_H_INCLUDED

class dynArr
{
    private:

        int *data;
        int size;

    public:
       dynArr(int);
       ~dynArr();
       void setValue(int, int);
       int getValue(int);
};

#endif // DYNARR_H_INCLUDED
```

dynarr.cpp

```
#include "dynarr.h"
#include <iostream>
using namespace std;

dynArr::dynArr(int s)
{
    data = new int[s];
    size = s;
}
dynArr::~dynArr()
{
    delete [] data;
}
int dynArr::getValue(int index)
{
    return data[index];
}
void dynArr::setValue(int index, int value)
{
    data[index] = value;
}
```

Solution Task 1:

dynarr.h

```
#ifndef DYNARR_H_INCLUDED
#define DYNARR_H_INCLUDED

// Template declaration
template <typename T>
class dynArr {
private:
    T* data;
    int size;

public:
    dynArr(int);
    ~dynArr();
    void setValue(int, T);
    T getValue(int);
};

#endif // DYNARR_H_INCLUDED
```

dynarr.cpp

```
#include "dynarr.h"
#include <iostream>
using namespace std;

// Constructor template implementation
template <typename T>
dynArr<T>::dynArr(int s) {
    data = new T[s];
    size = s;
}

template <typename T>
dynArr<T>::~dynArr() {
    delete[] data;

template <typename T>
void dynArr<T>::setValue(int index, T value) {
    if (index >= 0 && index < size) {
        data[index] = value;
    }
}

template <typename T>
T dynArr<T>::getValue(int index) {
    if (index >= 0 && index < size) {
        return data[index];
    }
}

// Explicit template instantiations for the types you want to use
template class dynArr<int>;
template class dynArr<double>;
```

main.cpp

```
#include "dynarr.h"
#include <iostream>
using namespace std;

int main() {
    dynArr<int> arrInt(5);

    arrInt.setValue(0, 10);
    arrInt.setValue(1, 20);
    cout << "Integer Array: " << arrInt.getValue(0) << ", " <<
arrInt.getValue(1) << endl;

    dynArr<double> arrDouble(3);
    arrDouble.setValue(0, 3.14);
    arrDouble.setValue(1, 2.71);
    cout << "Double Array: " << arrDouble.getValue(0) << ", " <<
arrDouble.getValue(1) << endl;

    return 0;
}
```